Module 19-1

Functional Design Challenges (1)

cadence

In this section, we're talking about the challenges with functional design.

We've already learned a little bit about the different kinds of hard hardware constructs that we see and the different levels of abstraction at which we have to write code.

In order to describe those, this section is going to talk about some of the problems you'll find.

Objective

In this module, you will:

- Identify the physical factors that are present challenges to functional design.

Topics include:

- Recap on scale and complexity of IC design
- X-Propagation
- Clock domain crossing
- Reset domain crossing

Your objective is to get started using SystemVerilog to describe the behavior of a digital design. To do that, you need to know some fundamental SystemVerilog language constructs.

Mainly they're related to physical factors that present challenges to the functional design.

Although we're coding at the lower level of abstraction register transfer level, we can't ignore the physical factors which will occur later on in the process.

So, we can see here a list of the topics we talk about.

And following this will be an introduction to low-power schemes, which use a language known as UPF.

Recap On Where We Are

We have seen how to create functional building blocks from boolean gates and flip-flops (FFs).

- For example, multiplexors, adders and so on.

An idea of scale:

- It takes 4 transistors to make a boolean NAND gate.
- It typically takes 18-20 transistors to make a FF.
- A complex processor chip might have 19 billion transistors.

cadence

For describing power, architecture, and power. We've seen how to create these functional building blocks from boolean gates and flip-flops.

These are all registers of how we describe multiplexes and others and multipliers and so on.

So, to give some idea of the scale of how this actually works in the physical world, it takes four transistors to make a boolean NAND gate.

And the brilliant Northgate, for that matter. In order to make a flip-flop. It takes something in the order of 18 to 20 flip-flops, depending on what kind of flip-flop it is.

We've already seen that's a complex process and might have anywhere approaching 20 billion or 30 billion transistors.

How Can We Fill a Chip?

By defining a huge number of boolean gates, FFs, latches, memory cells, and so on.

We would not have time to manually create schematics with FFs and boolean gates.

- It's been well over 30 years since IC design was done like that.

We have to RTL (Register Transfer Level) code.

EDA tools translate (synthesize) our RTL code into a schematic.

The complexity of the design requirements is truly mind-boggling.

However, writing the huge amount of RTL code required to define the desired functionality of our design is not the only concern.

cadence

That's a huge number, of course. So how can we actually fill this space? How can we create a design that contains that many transistors?

Well, we can't do it manually using schematics. Drawings, because it will take too long to do so.

The design of silicon IC used to be done like that years ago because that was the only option.

But now, because you can get so many more transistors into the same area.

Exponential increase following Moore's Law.

Now in 30 years, the increase is absolutely massive, and it's just too big. You cannot possibly create a schematic that would fill today's chips.

So, what we have to do then is write code at the registered transfer level, RTL level, and we're going to explain that in very shortly in the forthcoming lectures.

And what our tools do, our electronic design automation is what an IIDA stands for.

This translates our code into a schematic effectively because underlying all of this will be a schematic created on whatever target technology we're using, whatever silicon process we're using.

So, the complexity of the design requirements is truly mind-boggling when you think about what is a graphics processor required to do or what is a general CPU required to do.

Just stating that a huge amount of RTL code is required to fulfill that functionality is not our only problem.

Remember What's Important to Optimize in an IC

PPA

- The trade-off between power, performance, and area.

We may choose to address some of these aspects in our RTL code.

We now have to consider more than just the functional requirements specification when implementing RTL.

Power

- For example, describe clock gating at the RTL level using Gray coding for FSM's state encoding.

Performance

- For example, choose to re-code, in RTL, one slow implementation of a multiplier into a faster (more gates, more parallelism and less delay) implementation.

Area

- For example, share resources, like an adder, in the time domain to reduce overall transistor count.

5

cadence

This is the topic of this lecture. So forecast our minds back to when we talked about what's important in an IC economically. It's always a trade-off.

Everything is a trade-off between power, performance, and area. We can address some of those aspects with how we write our RTL code. When we're writing RTL code, we're not just considering what the functional requirements are. We're also thinking about some of these PPA, which are physical aspects of the design. So, for example, how do we address power?

We could describe clock gating at the RTL level and use different kinds of encoding for state machines. When creating multipliers, there are different architectures and multipliers we can create, which will have fewer changes or fewer gates in order to have lower power consumption. This all has to be handcrafted manually. So, for performance, we might choose to recode an RTL with the different implementations of a multiplier. You can't just say multiply by B if you want a power-efficient design; you have to go manually and create different kinds of multipliers depending on what your requirements are for the area.

We can share resources like memories and others and so on in the time domain in order to reduce the overall transistor count. So all these things are pulling in different directions, and we've got to find the sweet spot in order that we get the most economical design.

What Physical Issues Might Affect Our RTL Code?

Our RTL code is defined by the desired functionality of our IC in a software-like style.

However, we cannot write RTL without being aware of physical issues.

- One doesn't have this concern when writing "normal" software.

These are related to the underlying physical technology, namely:

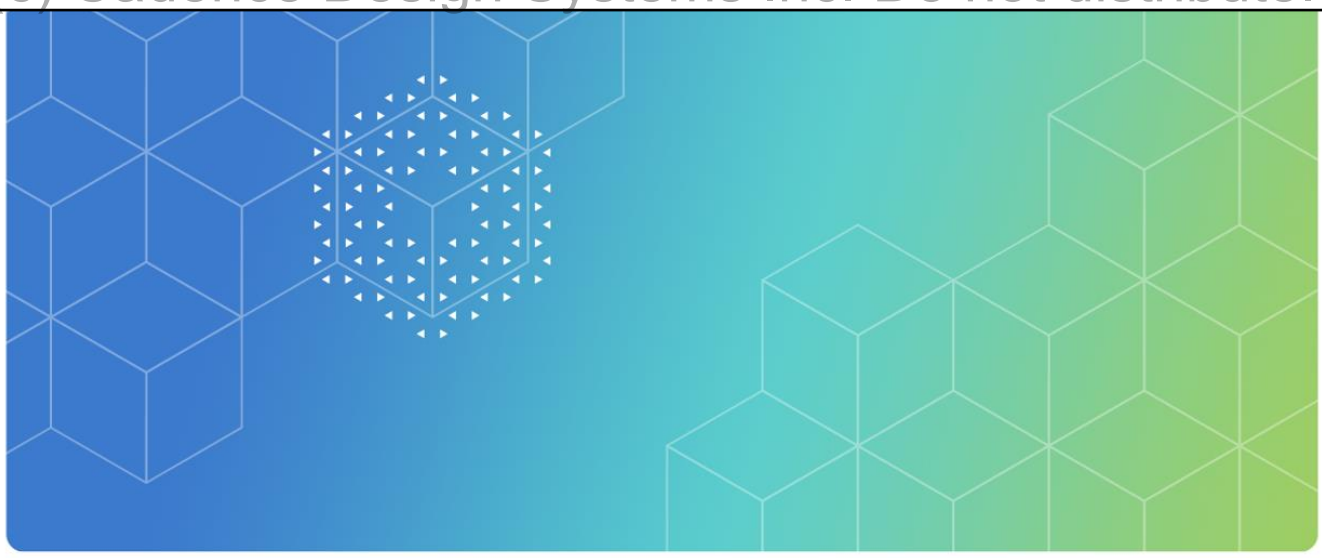
- **X-Propagation** – Unpredictable behavior due to timing and physical issues.
- **Clock Domain Crossing** – Unpredictable behavior due to timing and physical issues.
- **Reset Domain Crossing** – Unpredictable behavior due to timing and physical issues.
- **Low-Power Considerations** – Some aspects of low power we have to code into our RTL.

cadence

What physical issues might affect the code we write down? The code is, is software like it isn't software; of course, it's software-like. Because it's actually describing real-life hardware which has to care about things that software doesn't like concurrency, for example, time delays and glitches and all these kinds of things, we can't just write RTL and be unconcerned about those fiscal issues. If you're writing normal software, you don't have that consideration. You can just focus on the functional requirements.

There are some things related to the underlying physical technology. Now you have to create a chip in real life. These things are X propagation. So that is unpredictable behavior due to timing and physical issues: clock domain, crossing, reset, domain crossing, and other low-power considerations.

Some aspects of low power we can address in RTL. However, we said previously that UPF language, which describes low power intent, is another aspect of design we have to take care of, and we talk about UPF later. We're not going to describe these things now.



Module 19-2

X- Propagation

cadence

This page does not contain notes.

Objective

In this module, you will:

- Identify the different aspects of X-Propagation.

cadence

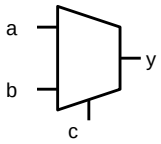
This page does not contain notes.

What Is an X-Value?

- In simulation: Value not known (could be 0 or 1).
- In synthesis: As a literal, designates value as “don’t care” (make it 0 or 1).

Where does the X value come from?

- Initial uninitialized value – a flip-flop without a reset.
- Unresolvable conflict of equal-strength 0 and 1 values.
- Unknown inputs.
- Deliberate assignment in RTL code:
  - In design code to indicate “don’t care.”
  - In verification code to indicate an error.




```
// RTL X-optimism
if (condition)
 result=a; // if c true
else
 result=b; // if other
```

9

cadence

- The SystemVerilog language designates the “X” literal to mean an unknown value.
- The unknown value initially appears on 4-state variables until the simulation assigns another value. During simulation, the unknown value can appear on nets simultaneously driven to equal-strength conflicting values and can appear on variables when so assigned.
- RTL simulation can mask those “X” values by converting them to another value, typically 0 or 1. This is because the synthesizable RTL branch statement has only “true” and “other than true” branches and no branch specifically for an “unknown” condition. The term for this is “X-Optimism.”

Documentation: *Elaborating Your Design Starting with X-Propagation*

The first question to ask is, what is an X value? Then we thought digital hardware was zeros and ones, but it’s not real. Life is not that simple. An X value is an unknown value in simulation. The value could be a zero, or it could be one we don't really know. We can't be sure in synthesis, which is the act of taking the RTL code and building this netlist from whatever technology library we're using. This is used as I don't care, i.e., the synthesis tool can make it a zero or one just to make a more efficient design.

So, it means different things depending on which tool you’re using. You might wonder, where does this come from then? How can we have a simulation where we have a value that isn’t a zero or one? Well, it can come from various places, typically on an initialized value, like a flip-flop without a reset unresolvable conflict between two equal-strength drivers. You’re driving a wire from two different places, and they have different values and unknown values of inputs. Deliberate assignments in RTL code for different reasons. So, in the design code, it indicates don’t care.

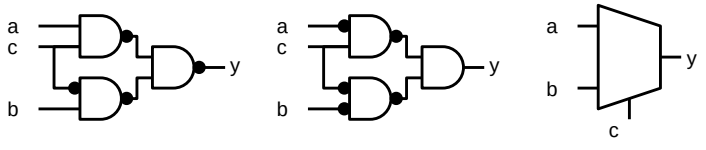
As we mentioned above, what we’re synthesizing in verification code, what we’re hoping is this X-propagates. And we see lots of different axes in our simulation, which tells us something's gone wrong. So how does this occur, then? If we have this condition here, if we're describing a mocks with this code here if this condition is true. The result is the value of A and the other condition. Then the result is B. If this condition gets an X value and an unknown value, the result will always be if we follow what this code says.

The X-Optimism Problem

X-Optimism

- RTL simulation can convert unknown values to known values.
- User debugs problems at the gate level with difficulty, which could be more easily debugged at RTL.
- Evaluation of synthesizable RTL branch statement has only “true” and “other than true” branches – no branch specifically for “unknown.”

```
// RTL
if ( c )
  y = a; //c true
else
  y = b; // other
```



Implementation A | Implementation B | Implementation C

a	b	c	:	y
0	0	x	:	0
0	1	x	:	1
1	0	x	:	0
1	1	x	:	1

Known values

a	b	c	:	y
0	0	x	:	0
0	1	x	:	x
1	0	x	:	x
1	1	x	:	x

a	b	c	:	y
0	0	x	:	x
0	1	x	:	x
1	0	x	:	x
1	1	x	:	1

Unknown values but 3 different results!

a	b	c	:	y
0	0	x	:	0
0	1	x	:	x
1	0	x	:	x
1	1	x	:	1

RTL simulations can mask unknown values by converting unknown values to known values. This X-Optimism can defer detection of design problems to a later post-synthesis simulation, where their debug is more difficult.

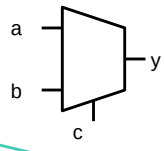
The illustration shows that the RTL model and three potential implementations all produce different output values when an unknown input value is given. The corollary *X-Pessimism* occurs when gate-level simulation produces an unknown value that the actual hardware does not produce. The first two implementations show this X-Pessimism with respect to the third. This is known as X-Optimism. So, if we have a control in our mocks, go back to the diagram here. This is the control for the mocks. If this is an x value, then the output merely follows B all the time. As we can see from this table, these are known values, which is overoptimistic because we don't really know what this value C is in real life. So, the dangers that are it can be difficult to debug problems at the gate level. This is when you've created this gate-level netlist from synthesis. It isn’t easy trying to correlate that gate-level description, that level of abstraction, with your RTL code.

Now what we’ve got in RTL simulation, if we just follow what that code says, is that we can convert unknown values into known values. For all the output here, Y is always a known value. Suppose we get any problems at the gate level. So, the gate level is a different level of abstraction. It's after we've taken our RTL and converted it to this gate-level netlist for synthesis. It's harder to debug that way. RTL is easier to debug.

So, any evaluation of the branch here, we're evaluating that. The branch statement is only true, and something other than true, including X. There isn't a branch for what happens if the book selection is an X. Moreover, we might get more different implementations in real life.

We might have three different implementations of this mark, all of which give us a different set of results given the same inputs if C is X. This is clearly a problem. The outcome depends upon the implementation which gets chosen.

X-Optimism Simulation Solution




```
// RTL
if ( c )
  y = a; // true
else
  y = b; // other
```

Known as X-Pessimism

Known as X-Optimism

Default

If the condition is unknown, assign the result as stated.

Only FOX guarantees that the simulation has no *new* unknowns.

```
// Default
a b c : y
0 0 x : 0
0 1 x : 1
1 0 x : 0
1 1 x : 1
```

Forward-Only-X (FOX)

If the condition is unknown, assign the result unknown.

```
// FOX
a b c : y
0 0 x : x
0 1 x : x
1 0 x : x
1 1 x : x
```

Compute-As-Ternary (CAT)

If the condition is unknown, assign the result as the hardware would.

```
// CAT
a b c : y
0 0 x : 0
0 1 x : x
1 0 x : x
1 1 x : 1
```

11

cadence

The Xcelium simulator solves the X-optimism problem by providing three modes in which to evaluate the branch statement when the condition is unknown.

- The simulator, by default, executes procedural assignments exactly as stated, treating the unknown condition as other-than-true.
- The simulator in forward-only-x mode, if the condition becomes unknown, assigns the unknown value to the variables. Only this mode can guarantee that the branching construct in later gate-level simulation produces no new unknown results.
- The simulator in compute-as-ternary mode, if the condition becomes unknown, it evaluates both the true and the false paths. It assigns the variables for the merged results from both paths. This is equivalent to the operation of actual hardware.

Why X's Are Dangerous

X's might mean that the behavior we observe in RTL simulation is not the same as seen in the real hardware.

X's may propagate to outputs.

- If one were purchasing design IP, then it would probably be a contractual requirement that this cannot happen.

X in clock gating logic or low-power control can hang a whole chip.

Makes simulation less deterministic:

- Different simulators produce varying results with X.
- X-Optimism, X-Pessimism issues.
- Gate-level simulation comparison with RTL simulation is hard with X in the design.

X can be misleading for code coverage and equivalence checkers.

cadence

So, why are these dangerous, then? Why don't we want this access? Access might mean the behavior we observe in RTL is not the same as the real hardware. Exhibit X might get two outputs.

Normally, if you're selling design IP to somebody, it's almost certainly a contractual requirement that access cannot propagate to outputs. Because when you integrate different IPs together, you can't guarantee something won't go wrong, something you didn't foresee.

If you have an X in clock gating or low-power control that can kill the whole chip, you can't recover from that with software fixes or metal layer fixes, and it makes simulation less deterministic because of all these different modes we have Fox and cats, and different simulators might produce different results.

With X, we must decide whether we use X-Optimism or Pessimism. Level simulation comparisons with RTL are hard with an X in the design. In addition to this, X can mislead code coverage checkers and equivalence checkers. These are formal kinds of proofs, so basically, they're bad news all around.

We Need to Make Some Choices

Can't we just reset all FFs so we don't get Xs?

That would be too costly in the area:

- Resettable FFs require more area than Non-Resettable FFs.
- It takes more space (area) to route the reset signal to each FF.

Which mode do I choose when simulating?

- CAT – Too Optimistic?
- FOX – Too Pessimistic?

Is there an alternative?

- Yes. We could use Formal Verification (algebraic proofs that don't require a testbench).

What's the difference regarding X-Propagation?

- In a simulation, each signal can only have one value at any given time – either 0 or 1.
- In formal, all possibilities are considered concurrently – both 0 and 1 at the same time.

cadence

We've got some choices to make here. Can't we just reset all the flip-flops so we don't get Xs in the first place? Well, the answer is no, we can't, because it costs too much in terms of area.

A flip-flop with the resets is bigger in terms of area than a flip-flop without one. You've got to root the reset wire to a reset, a flip-flop that takes more area on the chip as well. And an area was one of these trade-offs we make PPA; the E stands for the area.

In any case, what do I choose when simulating? Do I choose a cat or a fox? If there's a choice of two things, it tells you there is no kind of simple choice to make here. What's the alternative, then? Well, we could use formal verification. Formal verification uses algebraic proofs, which don't require a testbench. You know, the X-Propagation, whether it's a problem or not under all circumstances.

The difference between formal and simulation is that in simulation, each signal can only have one value at a given time, either zero or one in formal. All possibilities are considered. In formal, X means a set of values that could be zero or could be one, and simulation x is literally the discrete value one to be X. Although we're modeling digital hardware, which we expect to be zero or one, there is a different literal value, x, in formal something. If it's X, it means it can be the value zero or the value one.

What X Means in Formal

X does not exist in formal, just like in hardware.

- In the simulation, X is the discrete value 1'bX.

In formal, a signal is either 1'b0 or 1'b1.

If a signal's value is unknown, then it is a possible set of values `{0, 1}`.

- You can pass this through an inverter, which will be the set of values `{1, 0}`.
- It behaves exactly as the real hardware would.

The concrete value chosen will be the one that causes an assertion to fail or a cover to pass.

X-pessimism or X-optimism does not exist.

Cadence® JasperGold® has an X-Prop App dedicated to this exact problem.

The beauty of formal is that you know whether X will cause a problem under all circumstances.

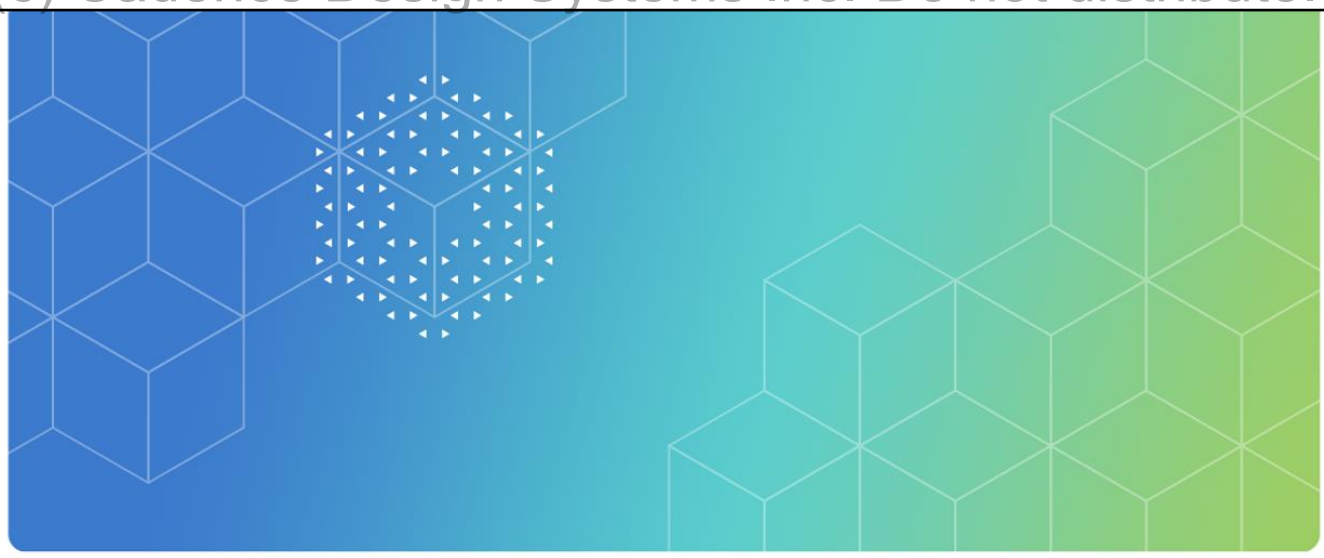
- Not just a particular set of simulations, with a particular CAT/FOX mode setting.

14

cadence

The cool thing about this is that if you pass that set of values through an inverter, you get the set of values one and zero. It behaves exactly like the real whole world and the concrete value you see, so it has to choose from this set what the actual value is. We find this out during a counter-example.

If an assertion fails or a cover statement passes, we get to see what the real value is. You don't get this concept of pessimism or optimism. It behaves like the hardware does. A formal tool from Cadence has an X-Prop app dedicated exactly to this problem because it's a problem that everybody has. And the beauty of that is, you know, whether X will cause a problem under all circumstances, not what you happen to simulate during all of your different tests with whatever setting you had, whether it's computers, ternary or forward, only X.



Module 19-3

Clock Domain Crossing (CDC)

cadence

Clock domain crossing is another aspect that is affected by the physical design itself.

Objective

In this module, you will:

- Identify the different aspects of Clock Domain Crossing in your design and its impact.

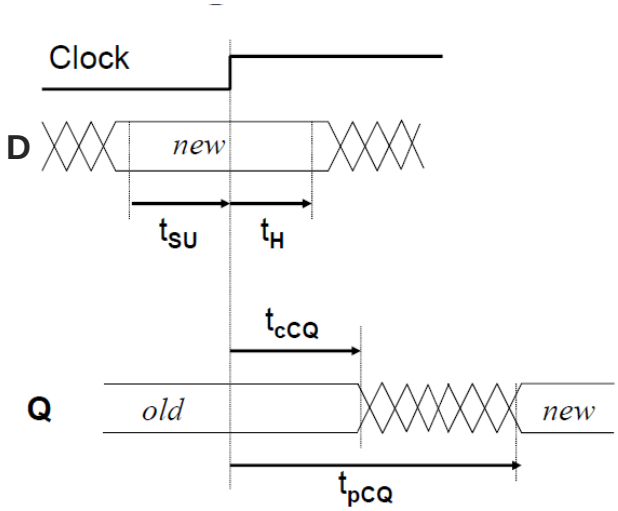
cadence

This page does not contain notes.

Timing Violations

FF Timing Specification

- Input data (D) must be stable.
  - Before active edge – setup time (t_{SU}).
  - After active clock edge – hold time (t_H).
- Propagation delay is the time from clock edge to the change in output (t_{pCQ}).



What happens when this specification is not followed?

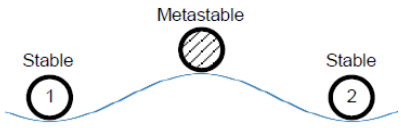
We've seen in a flip-flop that when the clock arrives, the data input can only change within a certain window. It cannot change between that line there and this line here. So the gap between the clock edge and the time prior to that where you can make the last change to date is known as setup time. And the gap between the clock edge and the earliest, at which you can then change the data in what is known as the hold time the propagation delays. The time from the clock edge to the output is actually changing.

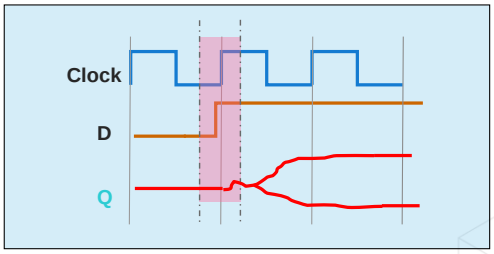
So, when you have a data input, the clock arrives, the output will bounce up and down for a bit before it settles to a new value. What happens if we don't follow that specification?

What Is Metastability?

When setup/hold conditions are violated, the output of a FF becomes unstable and unpredictable.

- Output may settle either way (1 or 0) after an unpredictable delay.
- This physical phenomenon is known as metastability.
- Seen for asynchronous inputs (clock to data relationship is unknown).
- In a gate-level simulation, a metastable value is represented as X.



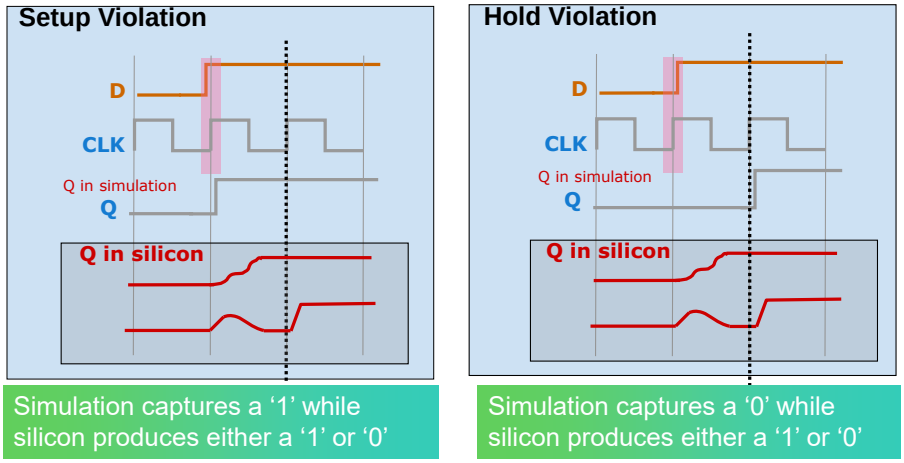


This is known as a timing violation.

A setup or hold time is violated. The output of the flip-flop becomes unstable. We can't predict whether it will be a zero or whether it will be one, or whether it will be somewhere in between. This is a physical phenomenon. This is due to the physical aspects of the chip. This is known as metastability.

Here, these are arriving too close to this rising clock edge that the red band represents the time we shouldn't change the value. It does change, however, and the output Q drifts off either to be a one or drift to be a zero after some time. So, this is a stable value. That's where that's represented as X inside of a simulation.

Metastability Modeling



Simulation captures a '1' while silicon produces either a '1' or '0'

Simulation captures a '0' while silicon produces either a '1' or '0'

Both the setup and hold effects must be modeled in order to detect all metastability related bugs.

cadence

How do we model these things? In the real silicone here, if we violate those set up at all times, we can see we might get different kinds of outputs.

We've got a model of those kinds of effects in simulation. If you can imagine at this next clock age if the signal is still drifting, be stable. In the sample here, one of the outcomes we might get a one a different outcome. We might get a zero, and we don't know which of these it will be. Same for a hold violation.

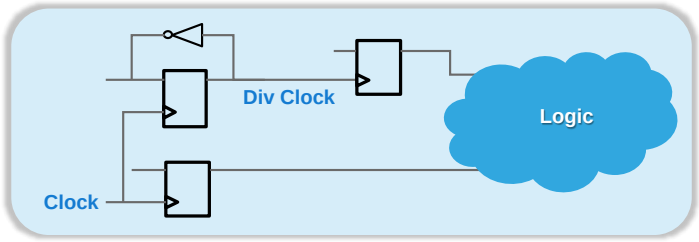
On the next clock, if we violate the whole violation at this age, on the next stage, we don't know what we're sampling necessarily. We could either sample a One or zero. And in real hardware, we don't know what we're going to sample.

What Is a Clock Domain?

A clock domain is defined as that part of the design driven by either a single clock or clocks that have constant phase relationships.

Synchronous and Asynchronous clocks

- **Synchronous:** Constant phase relationships
- **Asynchronous:** Variable phase and time relationships



cadence

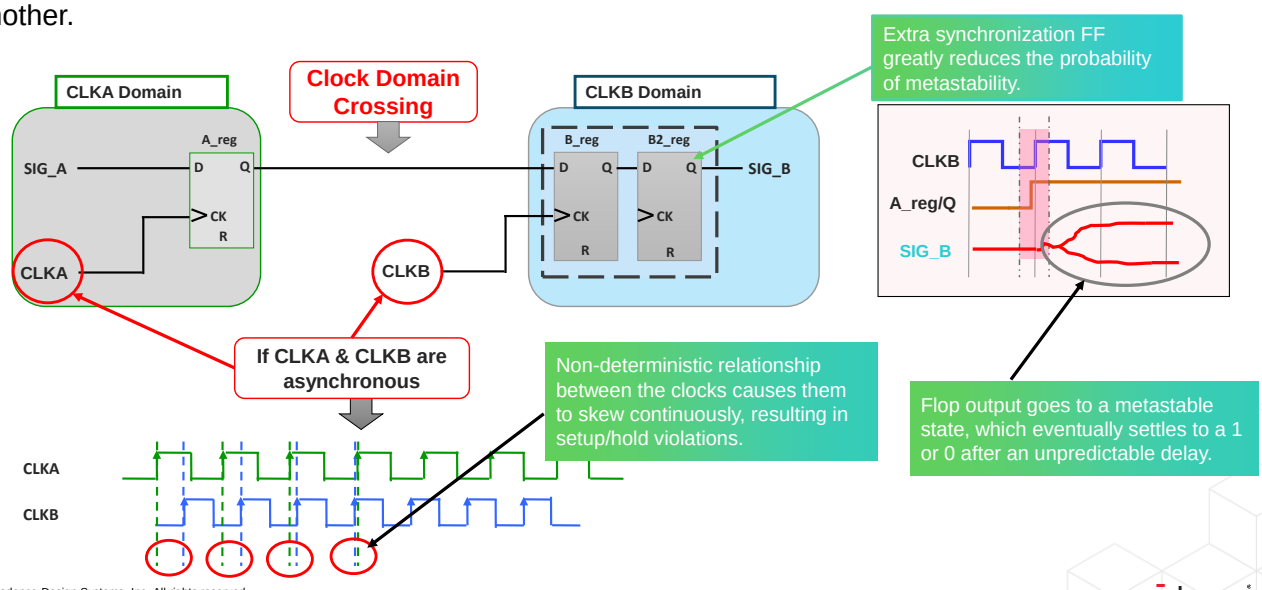
The subject we're talking about here is clock domain crossing.

What is a domain? It's part of the design driven by a single clock or clocks with a constant phase relationship.

You can get clocks that are synchronous to each other, i.e., they have a constant face relationship, or asynchronous, where they have a variable phase relationship.

Overview: Clock Domain Crossing

Clock Domain Crossing (CDC) occurs when a signal crosses from one asynchronous clock domain to another.



21

cadence

The clock domain crossing is when a signal crosses from one asynchronous clock domain to another. We've got two clock domains here. We've got a clock A and a clock B.

We're driving the output of this flip-flop, every Q edge, and sampling it. This is the D pots of the flip flop. We've got a different clock here. These clocks were drifting around.

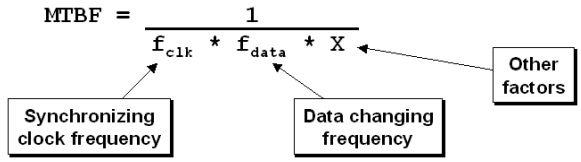
So here, these clocks are moving asynchronously to each other. Now at some point, we start narrowing that gap, and now we're changing the signals at the incorrect time. As far as the hold and set up times go and we're violating that requirement. Therefore, we will get an X value coming out of this signal B here. It's a signal B when the clocks get too close to each other. Therefore, we're violating setup and hold times. We're going to get this output going. That's stable.

So, what can do to mitigate this problem or greatly reduce the probability of stability is to put a synchronizing flip-flop here.

The likelihood of getting a signal, B, metal stability now is drastically reduced by orders of magnitude by having the synchronization circuit.

Mean Time Between Failure

- Metastability cannot be eliminated. Its effect can be minimized and reduce the probability of failure.
- The calculation of the probability of the time between synchronization failures (MTBF) is a function of multiple variables, including the clock frequencies used to generate the input signal and to clock the synchronizing flip-flop.



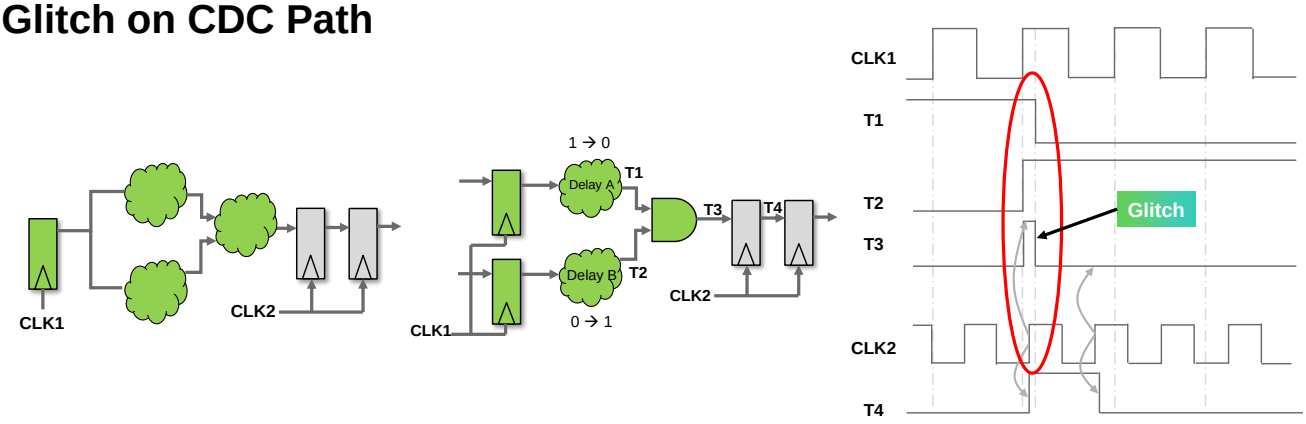
22 © Cadence Design Systems, Inc. All rights reserved.

cadence

Metastability is an effect that cannot be eliminated. We can minimize it by having different kinds of hardware structures to synchronize.

The probability is calculated via this equation here. So, it depends on the clock frequency, how often the data inputs, the flip-flop changes, and other factors as well to deal with the specific technology we're using.

Glitch on CDC Path

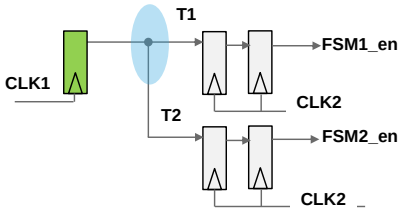


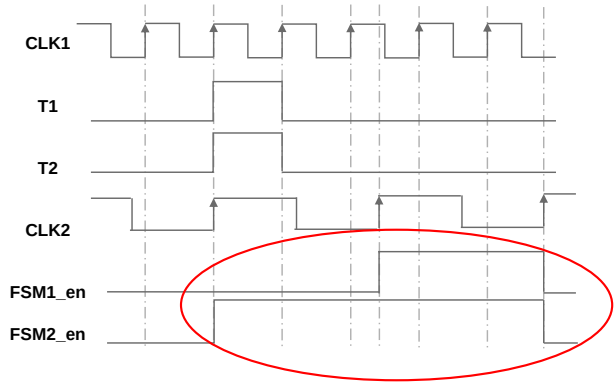
- Any logic in the crossover path can cause glitches and create functional errors downstream.
- Different delays in paths T1 and T2 causes a glitch.
- The glitch causes the flop output to go high when it was supposed to remain low.

cadence

This page does not contain notes.

Divergence of CDC Signal





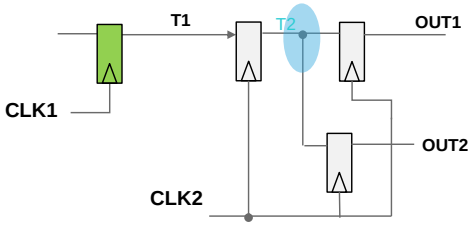
- A divergent logic style to multiple synchronization paths runs the risk of causing functional errors.
- Due to the propagation delay and different metastable settling times, the FSM1_en and FSM2_en could start at different times.
- This type of structure should be avoided by fanning out a single FSM enabled after synchronization to both FSMs.

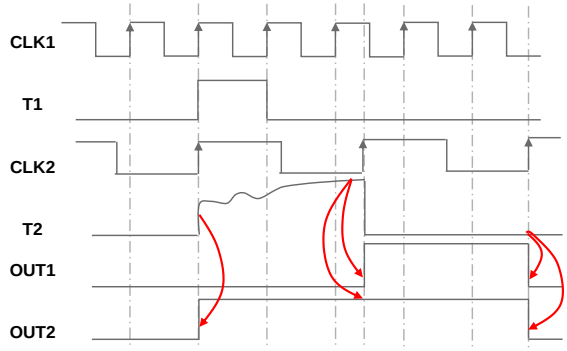
What we have here is an example of what happens if we try and diverge your signal. Here we've got an output of this flip flop from domain one, and we're splitting it off into two different synchronizes, both by clock two.

The signals from the first one enable an FSM to enable where our intent should be that these signals are both the same thing. However, due to different propagation delays and different settling times for these flip-flops, we might get a different value for the first one enabled and FSM to enable, which is what we intended, so we could avoid that kind of structure by fanning out a single FSM enables after synchronization, not before.

This is how we avoid that problem.

Divergence of Metastable Signals





OUT1 and OUT2 come out at two different clock edges as the settling and latching values of these metastable signals to the two flops are at different times.

cadence

We've got a signal from the main one where we're going through one flip-flop here, and output two is going to be stable, which means that although we're passing these through another synchronizing flip-flop here, the value that we will get from out one and out two may be different. Due to the different amounts of time it takes to settle in each different flip-flop.

We're storing the wrong value here at one or two or not storing the same value, which is our intent. So clearly, we shouldn't have taken that signal from here. We should have done it. But that signal should be taken from there instead.

Synchronization Schemes

- Metastable signals are unstable signals that need proper synchronization.
- We need to code, in RTL, an appropriate synchronizer that minimizes the possibility of an important signal becoming metastable.
- There are many ways of describing a synchronizer.
- We need to verify that our design does not have CDC problems.
- Structural analysis is basic CDC checking.
  - Advanced CDC analysis needs functional checks and metastability analysis.
  - Cadence JasperGold has an app dedicated to this problem.

cadence

In order to minimize the risk of meta stability and effect on the functionality of the design, we need proper synchronization circuits.

We need to code in RTL and appropriate synchronize that minimizes that possibility. There are many ways of describing these. We also need to verify that the design we created to mitigate this problem doesn't introduce any CDC problems or that there are domain crossings we forgot about. We didn't write synchronization.

What we have is JasperGold®. It is a formal verification tool with an app dedicated to these domain crossings to see if we get clogged domain crossing under any circumstance. Do we have an appropriate synchronization that will mitigate against all the problems we've seen about divergence and so on?

Commonly Used Synchronization Schemes

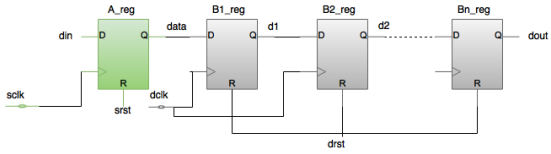


Fig: NDFF Synchronizer

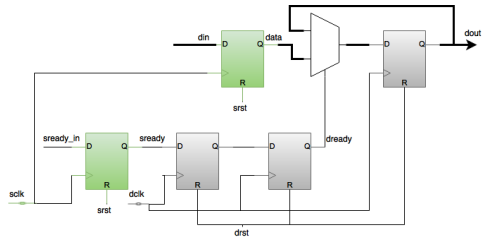


Fig: MUX Synchronizer

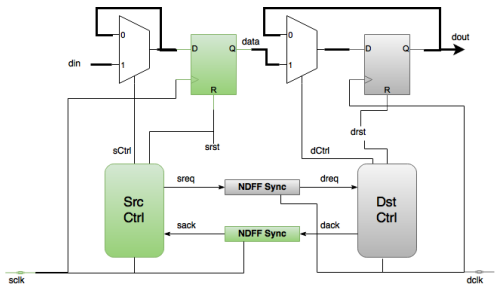


Fig: Handshake Synchronizer

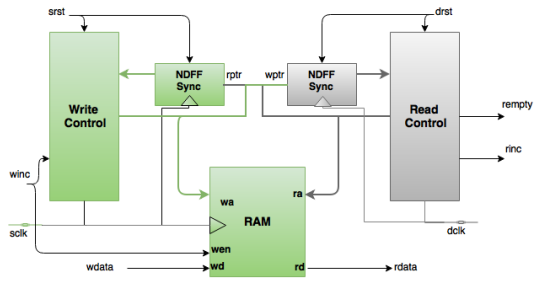


Fig: FIFO Synchronizer

27 © Cadence Design Systems, Inc. All rights reserved.

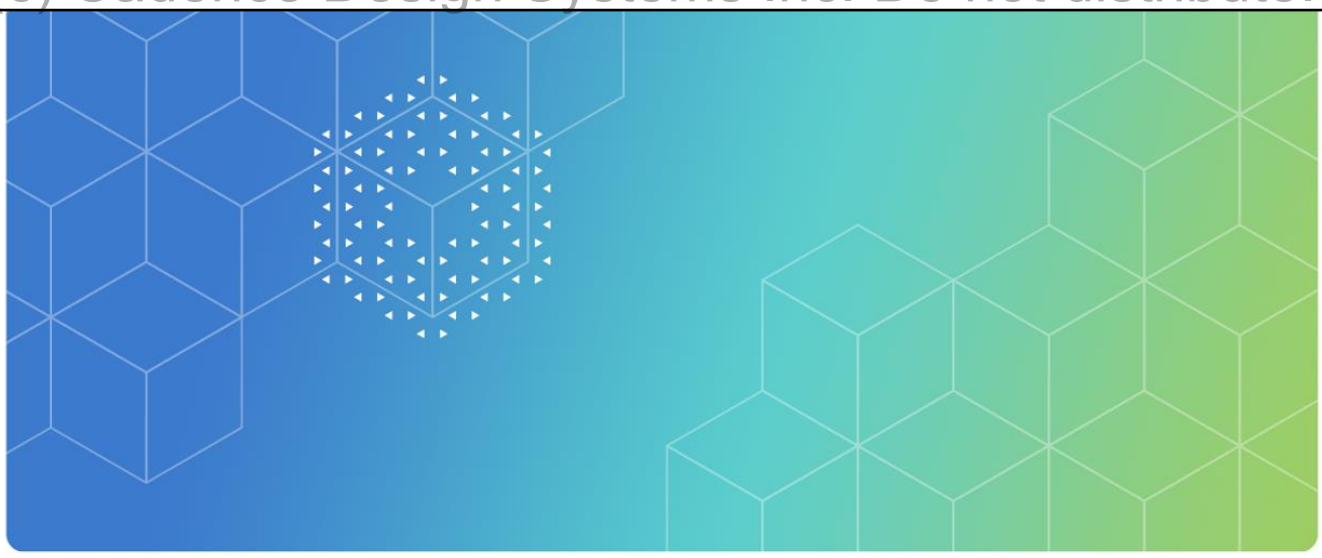
cadence

These are the different kinds of ones you get; you can see they range from quite simple ones.

Just from this clock domain SW clock, we have two or more flip-flops. That's why it's called an NDFF synchronizer until there are more flip-flops. To do each subsequent flip-flop, we put in here a synchronization flip-flop, reducing the probability of seeing stability.

Probability is drastically reduced on the second one, and it's reduced even further by a similar factor on the third one.

We can also have other constructs that use boxes, for example, on different handshakes. We have light controllers' handshaking between these two things and can have a user memory to synchronize. We have two port memories, where we can read and write with different clocks.



Module 19-4

Reset Domain Crossing (RDC)

cadence

This page does not contain notes.

Objective

In this module, you will:

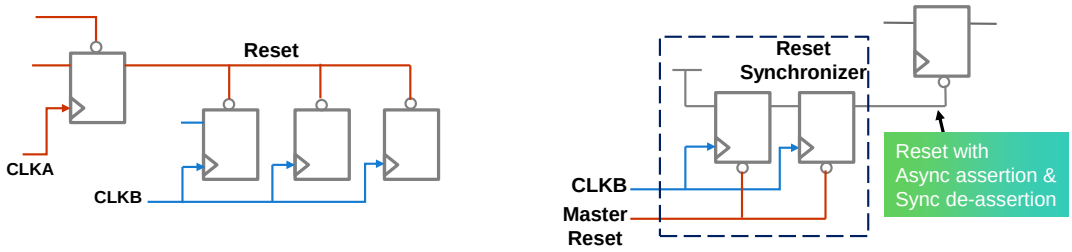
- Explore the concept of reset synchronization.

This page does not contain notes.

Reset Synchronization

If asynchronous reset is de-asserted near the active edge of the clock and violates the reset recovery time, it could cause the flip-flop to go metastable.

- A CDC verification tool, like the JasperGold CDC App, should report such scenarios.
- Solution: Asynchronous reset assertion and synchronous reset de-assertion.



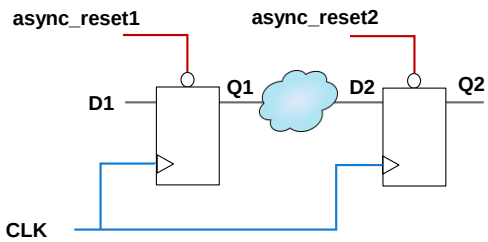
Now we have a similar problem with resets as well. This is called reset domain crossing because most modern chips do not have a single reset. They have more than one. So, if the reset is reasserted near the active edge of a clock, then we can violate the reset recovery time, which could cause the flip-flop to go to stable.

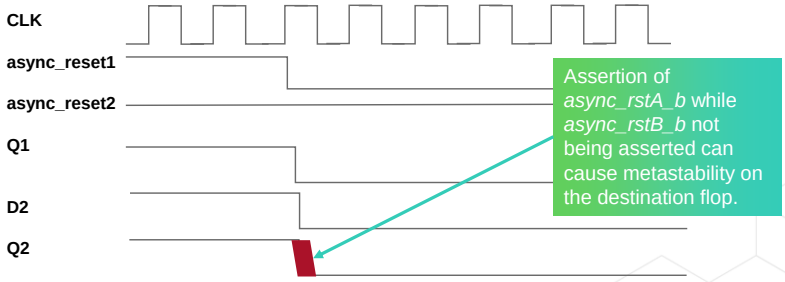
Then you may also have other physical factors like how long it takes the reset to propagate across the whole chip. We might get metastability out of these flip-flops if we have that kind of scenario. The solution is to use a CDC verification tool just for gold CDC to report such scenarios.

What we can do here is if we've got an asynchronous reset, we assert it asynchronously, and then we use a flip-flop to synchronize the reset here. The reset is asserted synchronously because we've got these synchronizing flip-flops here. There's the massive reset going in. The imports are tied high here. And we feed that into the reset of all these other flip-flops.

Reset Domain Crossing

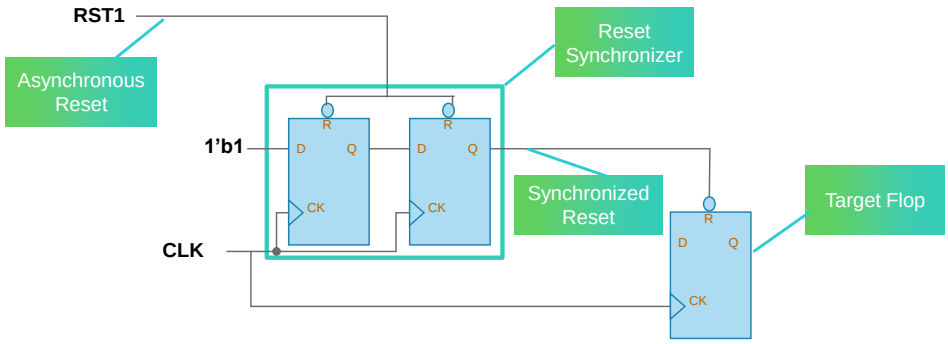
- Just like with clocks, there are reset domains in the design.
- All flops connected to the same reset belong to one reset domain.
- Reset domain crossing can happen even within the same clock domain.
- If the async reset of the source register is different from the async reset of the destination register, even though the data path is in the same clock domain, it can lead to metastability at the destination register.
- A CDC verification tool, like the JasperGold CDC app, should report such scenarios.





Just like with clocks, we have different reset domains, and all flip-flops are connected to the same reset signal. Have membership of one reset domain, with reset the main crossing. This can happen even if you have the same clock for the flip-flop. We have two different reset signals here, async reset one and asynchronous set two. And we might get undesirable effects from this, like metastability, when the async reset two was released.

Reset Synchronizer



- When asynchronous reset signal crosses from one clock domain to another:
  - It should pass through a reset synchronizer.
- The output of the reset synchronizer has an asynchronous assertion, and synchronous de-assertion property:
  - Provides additional clock cycles for the target flops to recover after reset removal.

cadence

Just like clock domain crossing, we need an appropriate reset synchronizer here.

We've got a code in RTL ourselves, and we will check that we've done it correctly with some kind of CDC app that will also check the reset domain crossings.

Here's our asynchronous reset.

These flip-flops are synchronized so that we apply the resets asynchronously, and we release the reset synchronously in order to not violate setup and hold times.

Recovery times cause metastability on the output of the flip-flop.

Summary

In these modules, we examined some of the Functional Design Challenges

- X-Propagation
- Clock domain crossing
- Reset domain crossing

In Functional Design Challenges part two, we will examine:

- Clock Gating
- An Introduction to Low Power design

Your objective is to get started using SystemVerilog to describe the behavior of a digital design. To do that, you need to know some fundamental SystemVerilog language constructs.

Mainly they're related to physical factors that present challenges to the functional design.

Although we're coding at the lower level of abstraction register transfer level, we can't ignore the physical factors which will occur later on in the process.

So, we can see here a list of the topics we talk about.

And following this will be an introduction to low-power schemes, which use a language known as UPF.